CHAPTER 4

Search Engineering: One Loop, Five Algorithms, Tested

Chapter at a glance

Before you start: Know what BFS, DFS, UCS, GBFS and A* each optimise (Chapters 2–3).
By the end, you can: explain why the five algorithms are one loop with a pluggable frontier; read and extend the `utils` package; write a unit test for a search function; run the toolkit from the command line; and design a benchmark whose numbers can be compared fairly.

Lab 3 turns the search code from Labs 1–2 into reusable software: a package with a common interface, hand-checkable tests, a command-line interface and a broader benchmark.

4.1 The pluggable frontier

The graph supplies states and successors; the query supplies the goal test. The frontier policy determines the algorithm (Figure 4.1).

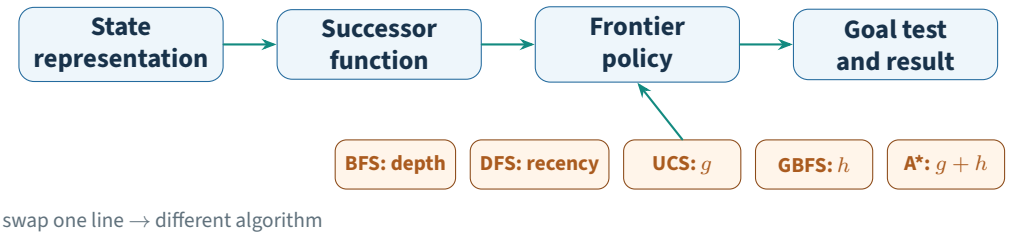


Figure 4.1. States, successors and the goal test come from the problem. The frontier policy determines the search algorithm.

In `utils/core.py`, BFS and DFS use `popleft` and `pop`, respectively. UCS, GBFS and A* wrap `best_first`, whose `mode` selects the priority. Weighted A* reuses it with the heuristic multiplied by 1.5.

Algorithm	Frontier structure	Entry and priority
BFS	`collections.deque`	state; oldest first
DFS	Python list as a stack	state; newest first
UCS	`heapq` min-heap	`(g, counter, state, g)`
GBFS	`heapq` min-heap	`(h(state), counter, state, g)`
A*	`heapq` min-heap	`(g + h(state), counter, state, g)`

4.2 One result type for every algorithm

Every algorithm returns the same frozen dataclass, allowing one benchmark loop to compare them:

```python
@dataclass(frozen=True)
class SearchResult:
    found: bool
    path: list
    hops: int
    path_cost_km: float
    expanded: int
    max_frontier: int
    runtime_ms: float

    def as_dict(self) -> dict:
        return asdict(self)
```

Three design decisions are hidden in those seven fields.

The cost is recomputed from the path. A helper sums edge weights after search, rather than using its stored g, keeping `path_cost_km` consistent with the returned path.

Failure has a shape. An unreachable goal returns `found=False`, an empty path, `hops=-1` and infinite cost. Callers check `result.found`.

Effort is part of the answer. `expanded` and `max_frontier` are counted inside the loop, so every comparison in Labs 1–3 comes from the same counters.

`as_dict()` converts a result into a pandas row via `{**result.as_dict()}`, as used in every lab metrics file.

4.3 Separating responsibilities

`utils` has five modules, and each knows nothing about the others' concerns (Figure 4.2).

Module	Responsibility
`core.py`	The five algorithms and `SearchResult`. Works on any `{state: [(neighbour, cost)]}` dictionary; never mentions airports.
`openflights.py`	Reads the two CSV files, cleans them, computes Haversine distances, returns the graph. Never searches.
`plotting.py`	Draws routes on longitude/latitude axes.
`__main__.py`	The command-line interface: parses arguments, loads the graph, calls one algorithm, prints.
`test_search.py`	The unit tests, on graphs of four to five states. Imports `core` only.

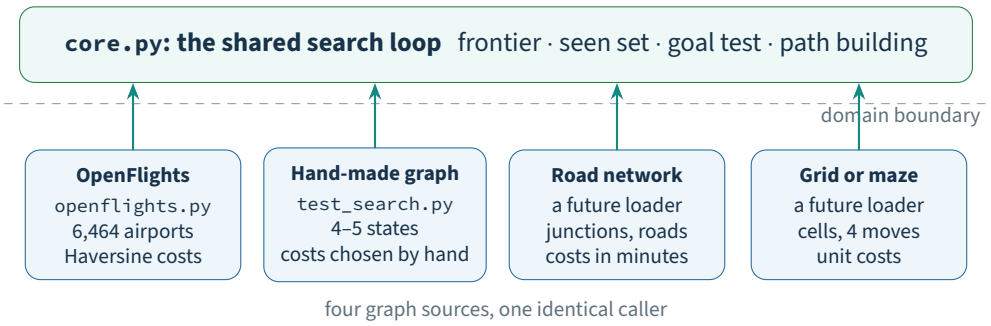


Figure 4.2. Graph-specific loaders below the dashed line produce `{state: [(neighbour, cost)]}` dictionaries for shared search code above it.

A road network needs a new loader, not new search code or tests. A new algorithm changes `core.py`, not the loader, CLI or benchmark. Lab 3 asks you to identify this boundary (Figure 4.2).

4.4 Priority queues and tuple ordering

Python's heapq pops the smallest tuple, comparing element by element from the left. The frontier entries in `best_first` are `(priority, counter, state, g)`, and the `counter` is there for a reason (Figure 4.3).

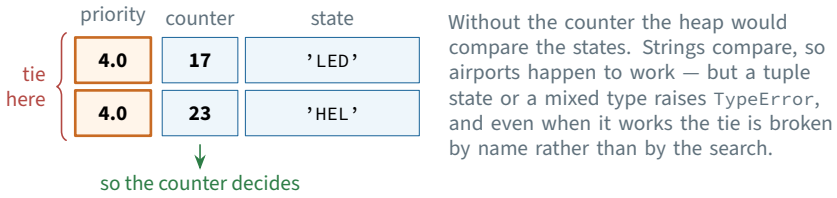


Figure 4.3. Heap tuples compare left to right. A unique counter breaks priority ties by insertion order, avoiding comparisons between states.

Common mistake

Placing a payload directly after the priority makes ties depend on payload comparisons and possibly state names. Insert a unique increasing counter to resolve ties by insertion order without comparing payloads.

4.5 Testing search code

A successful run on 6,464 airports can hide wrong costs, expansion counts or cycle handling. Tiny graphs make these errors hand-checkable. `utils/test_search.py` contains eight tests, one per failure mode:

Test	What a failure would mean
start equals goal	The goal test runs before any expansion; hops and cost are zero.
unreachable goal	Failure is reported as `found=False`, not an exception or a wrong path.
cycles terminate	The seen-set stops DFS from looping forever.
BFS and UCS optimise different objectives	The two algorithms give different answers on a graph built so that they must.
frontier update finds the cheaper path	A state first reached expensively is later replaced by a cheaper route.
A* matches UCS with a consistent heuristic	The optimality guarantee holds on a hand-checked graph.
GBFS is not assumed optimal	The test asserts the *expected* suboptimality rather than pretending it away.
negative cost is rejected	The precondition of UCS is enforced with a clear error.

One test in full shows the pattern — a tiny graph, a hand-derived expectation, one assertion:

```
def test_frontier_update_finds_cheaper_path():
    graph = {"A": [("B", 10.0), ("C", 1.0)], "C": [("B", 1.0)], "B": [("G", 1.0)], "G": []}
    result = ucs(graph, "A", "G")
    assert result.path == ["A", "C", "B", "G"]
    assert result.path_cost_km == pytest.approx(3.0)
```

These dataset-free tests import only `core`, run in under a second, and can run on any machine or continuous-integration server.

Key idea

Test the algorithm on the smallest graph that can expose the bug, and test the dataset loader separately. A test that needs real data to run tests both at once and diagnoses neither.

4.6 A command-line interface

`utils/__main__.py` makes the package runnable outside a notebook:

```
cd all_experiments
python -m utils --algorithm astar --start KUL --goal KEF
ASTAR: found=True, hops=4, cost_km=11461.3, expanded=69, runtime_ms=4.20
KUL -> HKT -> LED -> HEL -> KEF
```

The thirty-line module parses three options, loads the graph, selects an algorithm and prints its result. It calls existing search and cleaning code, reproducing Lab 2's answer. Lab 3 runs it as a subprocess and checks exit status 0.

4.7 Benchmarking fairly

Lab 3 benchmarks BFS, UCS, GBFS and A* on ten route pairs in `utils/search_queries.csv`:

Algorithm	Success rate	Mean cost (km)	Mean hops	Mean expanded
BFS	1.0	16,469	1.8	324
UCS	1.0	14,321	2.2	2,700
GBFS	1.0	15,087	2.1	2.1
A*	1.0	14,321	2.2	63

The notebook enforces four conditions for a fair comparison:

- **Same graph, same queries, same machine.** Every algorithm sees the identical cleaned graph and the identical ten pairs.
- **Success rate before averages.** Averaging only solved queries hides failures on hard cases. All four methods solve all ten queries here.
- **Effort in expansions, not milliseconds.** The runtime column is kept but is not the basis for conclusions.
- **Every path validated.** The notebook checks that A*'s cost equals UCS's on all ten queries and counts the queries on which GBFS is costlier (seven of ten).

For these kilometre-based queries, A* matches UCS at 2 % of its effort. Choose BFS for fewest legs and GBFS when reduced effort matters more than optimal cost.